



Optimization Strategies on Particle in Cell (ZPIC) in Supercomputer Deucalion Parallel Computing

Diogo Coelho da Silva
pg61444@alunos.uminho.pt

Tomás Alexandre Torres Pereira
pg59810@alunos.uminho.pt

Abstract—

IN this report, we present optimizations applied to ZPIC on Deucalion ARM [1] nodes. We performed both single-core and multi-core optimizations and observed that ZPIC operates in a gray area between memory-bound and CPU-bound performance. This is due to challenges with vectorization and cache locality within the application.

I. INTRODUCTION

This report demonstrates methods to enhance the performance of Particle-In-Cell (PIC) simulations, particularly ZPIC, on ARM nodes within the Deucalion supercomputer. Throughout this report, we focus primarily on examining the impact of multi-threading on the performance of ZPIC.

A. Environment

1) *Profiling tools*: For profiling, we used *perf* and the *Firefox Profiler*, the former for profiling hardware events and the latter for analyzing hotspots.

2) *Architecture*: Our work was developed exclusively on **one ARM node of Deucalion**. The ARM node used has an A64FX [2] CPU, which is very focused on vectorization. Therefore, we conducted an analysis on vectorization performance optimizations.

B. Methodology

To have a consistent and solid approach to optimizing ZPIC we followed a specific methodology. This process began with a preliminary assessment to identify architectural software-related bottlenecks, followed by a hotspot analysis to guide our optimization efforts. During our testing phase, we conducted multiple repetitions of the same test to ensure the results were consistent.

II. OPTIMIZATIONS FOR ZPIC ON A64FX

A. Single-Core and the Impact of Vectorization on A64FX

Since our work was developed on the ARM A64FX, vectorization plays a significant role. This occurs because, in this architecture, there are two vector pipelines (FLA and FLB) that are capable of floating operations (SVE [3] 512 bit wide).

To maximize vectorization performance, we relied on **Clang**, as its counterpart GCC is generally less effective at exploiting vectorization on SVE architectures. Since Clang already performs most of the heavy lifting, our main effort focused on restructuring the code to expose patterns that the compiler can reliably vectorize. Initially, most memory accesses followed an **Array of Structures (AoS)** layout. This access pattern does not take full advantage of the contiguous memory requirements that benefit vectorization. To address this limitation, we reorganized the data into a **Structure of Arrays (SoA)** layout, which exposes contiguous vectors of each field and therefore enables much more effective vectorization. To this extent, we created a **Float3Buffer** and a **ParticleBuffer**. With these optimizations, we achieved a speedup of 2.1x, and the number of retired SVE instructions went up from 0% to 23% of the total number of instructions. Within these results, we know that some vectorization possibilities might be missing and some code is inherently sequential, so vectorization is a ceiling on our performance analysis.

B. Multi-Core Scalability on A64FX

While vectorization provides substantial benefits for single-core performance, the A64FX is designed to deliver its strongest performance when **multiple cores** operate concurrently. In this section, we analyze how thread-level parallelism interacts and affects the performance of the simulation. In our optimizations, parallelism was introduced at the loop level inside the particle push (*spec_advance*), current filter (*kernel_x*), and field update (*emf_advance*) stages.

Among all stages, *spec_advance* dominated both **execution time** and **scalability behavior**. Not only does it contain the majority of Floating Point Operations (FPO's), but it is also responsible for highly irregular **memory access** patterns during particle-field interpolation and current deposition. The other main hotspot we found was *kernel_x* during the current smoothing phase.

On improving *spec_advance*, a naive implementation would be the use of `#pragma omp atomic` for each current deposition update per particle, which would serialize all the threads and, of course, destroy **scalability**.

To avoid synchronization overhead in the current deposition stage J_0 , we replaced `#pragma omp atomic` operations with a **block-partitioned** scheme, where the domain is divided into independent regions and each thread deposits currents exclusively within its assigned block. This fully prevents write conflicts and eliminates the need for atomics.

To remove serialization of *kernel_x*, we introduced a temporary buffer per thread that stores the current smoothing values per cell. In the end, we merge the results of all threads in J_0 buffer.

Despite the optimizations, scalability remains modest. As the number of threads increases, the **work per thread decreases**, OpenMP scheduling overhead grows, and the execution becomes increasingly dominated by **memory-bandwidth contention** rather than computation.

In addition, after analyzing hotspots, we observed that the cost of creating a new OpenMP team increased significantly once we exceeded 12 threads (the number of cores per CMG). This explains the degradation in scalability beyond that point. For

future implementations, a viable solution is to create all threads once in an outer parallel region that encloses methods executed many times, instead of repeatedly creating and destroying parallel regions inside each iteration. This reduces thread-team creation overhead and improves scalability.

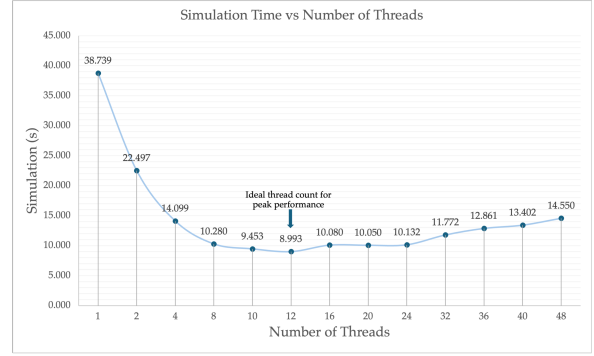


Fig. 1: Simulation time scalability of ZPIC across different thread counts.

The speedup is further limited by the **inherently sequential** portions of code. According to **Amdahl's Law** [4], these factors impose a strict upper bound.

In practice, multi-threading achieved a maximum **speedup of $4.31\times$ at 12 threads**, relative to the SVE-optimized baseline. This results reflects the fact that ZPIC is strongly **memory-bound**. On A64FX, the workload rapidly **saturates the HBM2 bandwidth**, and increasing concurrency beyond this point no longer improves performance.

Consequently, the optimal configuration for ZPIC on A64FX is not the one with the most threads, but the one operating just below the **HBM2 saturation point** (typically 8–12 threads). A detailed view of the speedup behaviour is provided in Appendix C.

III. CONCLUSION

By restructuring data structs, improving particle-buffer locality, and enabling SVE vectorization, we achieved up to **2.1x** single-core speedup. Multi-threaded performance, however, is constrained by the **memory-bound** nature of PIC algorithms and the **early saturation of HBM2**, causing scalability to plateau around **8–12 threads**.

Since PIC methods naturally support domain decomposition, an **MPI-enabled version** could scale efficiently across **multiple nodes**, making it a promising direction for future work.



REFERENCES

- [1] F. F. para a Computacao Científica Nacional, “Macc – plataforma de documentação fccn,” 2025, available online <https://docs.macc.fccn.pt/>. [Online]. Available: <https://docs.macc.fccn.pt/>
- [2] F. Limited, *FUJITSU Processor A64FX Datasheet*, Fujitsu, 2023, available at https://www.fujitsu.com/downloads/SUPER/a64fx/a64fx_datasheet_en.pdf. [Online]. Available: https://www.fujitsu.com/downloads/SUPER/a64fx/a64fx_datasheet_en.pdf
- [3] A. Ltd., *Scalable Vector Extensions (SVE)*, 2024, available online <https://developer.arm.com/Architectures/Scalable%20Vector%20Extensions>. [Online]. Available: <https://developer.arm.com/Architectures/Scalable%20Vector%20Extensions>
- [4] E. ScienceDirect Topics, “Amdahl’s law – an overview,” *ScienceDirect Topics on Computer Science*, 2024, available online <https://www.sciencedirect.com/topics/computer-science/amdahls-law>. [Online]. Available: <https://www.sciencedirect.com/topics/computer-science/amdahls-law>

APPENDIX A

A64FX ARCHITECTURE

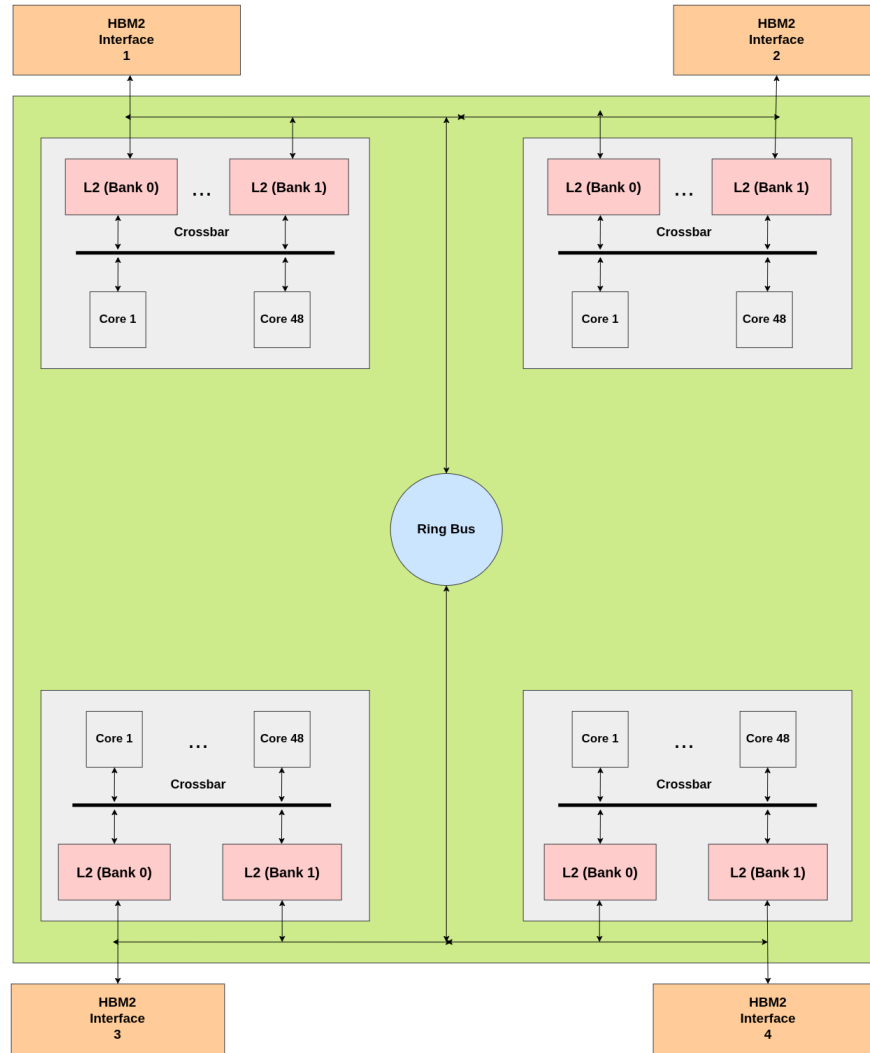


Fig. 2: Architectural view of A64FX

APPENDIX B

SINGLE-CORE AND SVE OPTIMIZATIONS

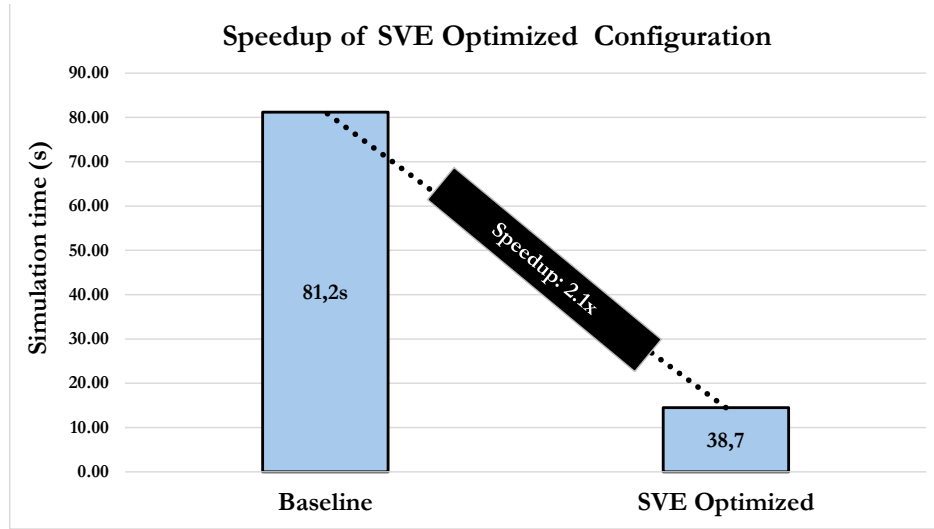


Fig. 3: SVE Single-Core optimizations speedup

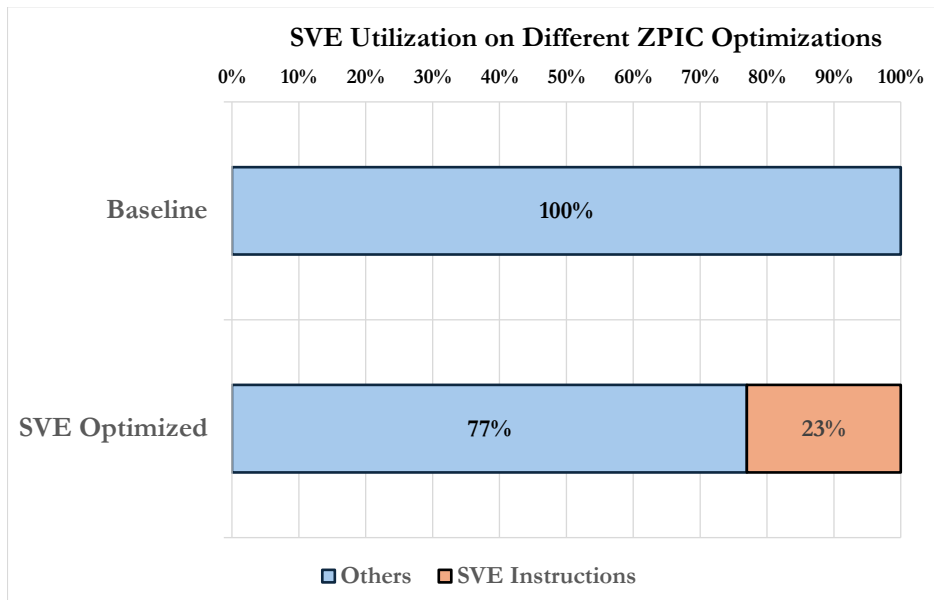


Fig. 4: Distribution of SVE retired instructions on different optimizations

APPENDIX C

MULTI-THREAD OPTIMIZATIONS

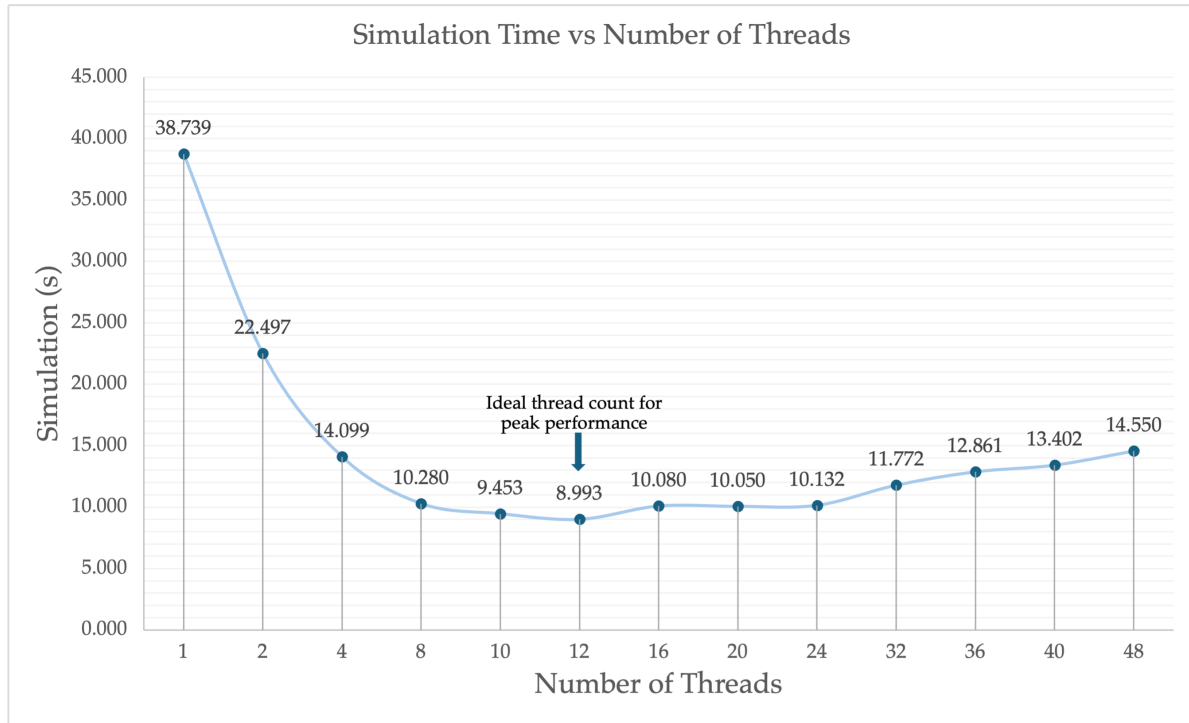


Fig. 1: Simulation time scalability of ZPIC across different thread counts.

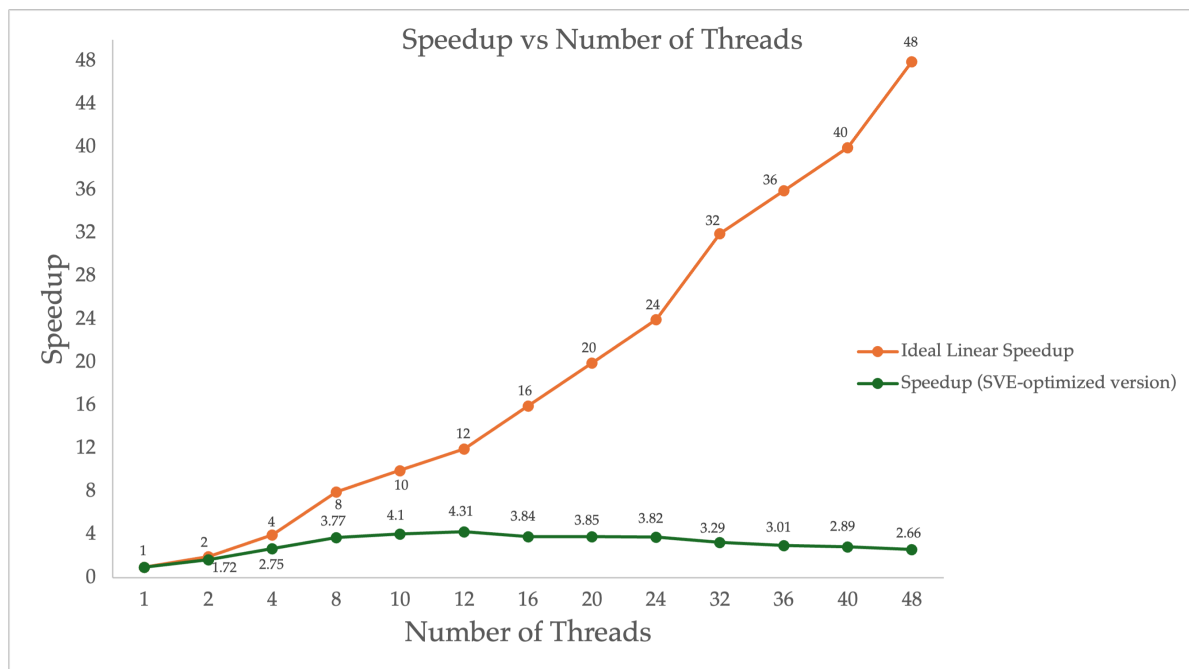


Fig. 5: Strong scalability analysis based on SVE-optimized version of ZPIC.